Documentação comparativa de APIs de QR Code

Pesquisa • vantagens e desvantagens • escolha • plano de testes assistido por IA

Objetivo

Comparar três APIs que geram QR Codes, selecionar a mais adequada para um projeto simples e registrar testes com um conteúdo igual para todas: “Teste de API QR Code”. A documentação oficial foi consultada para confirmar endpoints e parâmetros.

1. APIs pesquisadas

A) GoQR / QRServer

Documentação: goqr.me/api/doc

Gera uma imagem por uma requisição HTTP simples, usando o conteúdo e o tamanho como parâmetros. O uso básico não exige chave de API. Também oferece uma API separada para leitura de QR Codes.

Vantagens: integração simples; uso básico sem chave; parâmetros para tamanho, cores e formato; também pode ler QR Codes.

Desvantagens: menos recursos de gerenciamento que uma plataforma dinâmica; conteúdo enviado em GET precisa estar codificado corretamente; uso intenso pode ser limitado se o tráfego for considerado abusivo.

B) QuickChart QR API (estática)

Documentação: quickchart.io/documentation/qr-codes

Gera imagens PNG, SVG, JPG/JPEG ou Base64. Permite configurar tamanho, margem, cores e estilo. O endpoint estático não precisa de chave para o exemplo básico.

Vantagens: personalização visual; vários formatos de saída; aceita GET ou POST; útil para sites e aplicações.

Desvantagens: há mais parâmetros para aprender; códigos estáticos não permitem trocar o conteúdo depois de impressos; estilos e logotipos precisam ser testados com leitor real.

C) QRCode Monkey API

Documentação oficial: qrcode-monkey.com/qr-code-api-with-logo

API voltada à criação de QR Codes personalizados, com opções de design, cores e logotipo. A documentação informa formatos como PNG, SVG, PDF e EPS. O acesso à API é oferecido por meio do RapidAPI.

Vantagens: personalização avançada; suporte a vários formatos; adequada para códigos com identidade visual e logotipo.

Desvantagens: exige configuração de acesso pela plataforma RapidAPI; autenticação e limites dependem do plano; mais passos de configuração que uma URL pública simples.

2. Comparação resumida

Critério	GoQR / QRServer	QuickChart	QRCode Monkey
Facilidade	Alta	Alta	Média
Personalização	Básica a boa	Boa	Avançada
Autenticação	Não no uso básico	Não no endpoint básico	RapidAPI / chave
Ponto forte	Simplicidade e leitura	Flexibilidade de parâmetros	Design e logotipo
Melhor cenário	Projeto inicial	Site ou protótipo	Material visual personalizado

3. API escolhida: QuickChart

Motivo da escolha: para um projeto acadêmico ou protótipo, a QuickChart oferece o melhor equilíbrio entre simplicidade e personalização. A requisição básica não exige chave, pode ser testada diretamente no navegador e permite controlar o tamanho e o formato da imagem. A documentação também mostra exemplos por GET e POST, o que facilita a evolução do projeto.

Por que não as outras? GoQR é igualmente simples, mas a QuickChart tem opções de estilo e formato convenientes para demonstrar personalização. QRCode Monkey é mais forte em design avançado, porém exige configuração de acesso via RapidAPI, desnecessária para um primeiro teste.

4. Testes assistidos por IA

Caso de teste comum: gerar um QR Code contendo exatamente o texto “Teste de API QR Code”. O critério de sucesso é receber uma imagem de QR Code que, ao ser lida por um celular, devolva exatamente esse texto.

Teste 1 — GoQR / QRServer

Abrir requisição de teste: Gerar QR Code no GoQR

Procedimento: abrir o link; confirmar que aparece uma imagem; escanear com a câmera do celular; conferir se o texto retornado é “Teste de API QR Code”.

Resultado documentado: a URL segue o formato oficial `create-qr-code` com `data` e `size`. A documentação confirma que a API retorna uma imagem PNG para esse tipo de chamada. A leitura final com celular deve ser feita pelo usuário para comprovar a decodificação na prática.

Teste 2 — QuickChart

Abrir requisição de teste: Gerar QR Code na QuickChart

Procedimento: abrir o link; confirmar que aparece uma imagem; escanear com a câmera do celular; conferir se o texto retornado é “Teste de API QR Code”.

Resultado documentado: a URL segue o endpoint oficial `/qr`, com `text`, `size` e `format=png`. A documentação confirma esses parâmetros. A leitura final com celular deve ser feita para validar o QR gerado.

Teste 3 — QRCode Monkey

Documentação e acesso à API: QRCode Monkey API

Procedimento: entrar no RapidAPI, abrir o produto oficial Custom QR Code with Logo e configurar a autenticação conforme as instruções; enviar uma requisição POST para `/qr/custom` com o corpo abaixo; guardar a imagem retornada e escaneá-la.

```
{
  "data": "Teste de API QR Code",
  "size": 300,
  "file": "png",
  "download": false,
  "config": {}
}
```

Resultado esperado: uma imagem PNG contendo o texto informado. A API exige acesso autenticado pela plataforma RapidAPI; portanto, sem uma chave válida não é possível concluir o teste de ponta a ponta. A chave não deve ser colocada em código público nem compartilhada com a IA.

5. Registro honesto dos resultados

A IA foi usada para localizar e conferir a documentação, comparar recursos, preparar as requisições e definir os critérios de validação. Neste ambiente, não foi possível confirmar diretamente as respostas binárias das APIs nem escanear as imagens com a câmera de um celular. Por isso, os testes GoQR e QuickChart estão preparados em links diretos e o teste QRCode Monkey está descrito para execução com chave. Não se afirma que os três testes foram concluídos fisicamente.

Checklist para finalizar os testes

☐ Abrir o link do GoQR e confirmar que a imagem aparece.

☐ Abrir o link da QuickChart e confirmar que a imagem aparece.

☐ Executar a requisição do QRCode Monkey com autenticação válida.

☐ Escanear cada imagem com o celular e confirmar o mesmo texto.

☐ Registrar o resultado real: aprovado, falhou ou bloqueado por autenticação.

6. Conclusão

A QuickChart foi escolhida por facilitar o primeiro teste e oferecer personalização sem exigir autenticação no endpoint estático básico. GoQR / QRServer é uma alternativa muito simples e também disponibiliza leitura de QR Codes. QRCode Monkey é mais indicada quando o foco principal é a aparência personalizada, mas exige mais configuração para acessar a API.

Fontes oficiais

- GoQR / QRServer — documentação
- QuickChart — documentação de QR Codes
- QRCode Monkey — documentação da API

Sprint 2 — QR-03: Prova de conceito e relatório de teste

Período: 23/10/2026 a 05/11/2026 Prioridade: Alta
Código local: QR-03 Origem: issue #10
Situação planejada: planejada (execução parcial registrada neste relatório)

1. Objetivo e escopo

Validar tecnicamente a alternativa escolhida, QuickChart QR API, gerando um QR Code a partir de uma URL fictícia e segura. O conteúdo utilizado foi https://example.com/qr-03; nenhum dado pessoal real foi inserido.

2. Integração escolhida

Foi mantida a QuickChart por ter um endpoint HTTP simples, permitir configurar tamanho e nível de correção de erros e gerar PNG. A documentação informa os parâmetros text, size, format e ecLevel (L, M, Q ou H). Documentação oficial da QuickChart.

Requisição de referência para a API:

Abrir QR Code de teste na QuickChart (300 px, correção Q)

```
GET https://quickchart.io/qr
  ?text=https%3A%2F%2Fexample.com%2Fqr-03
  &size=300&ecLevel=Q&format=png
```

3. Código mínimo de integração

```
import requests

url = "https://example.com/qr-03"
params = {
    "text": url,
    "size": 300,
    "ecLevel": "Q",
    "format": "png"
}
resposta = requests.get(
    "https://quickchart.io/qr",
    params=params, timeout=30
)
resposta.raise_for_status()
open("qr03.png", "wb").write(resposta.content)
```

4. Testes de tamanho e correção

Foram preparados três casos com a mesma URL. A leitura foi verificada por dois decodificadores de software independentes: OpenCV QRCodeDetector e ZBar (via pyzbar).

Caso	Tamanho	Correção	OpenCV	ZBar	Conteúdo lido
1	200 × 200 px	M	Aprovado	Aprovado	URL exata
2	300 × 300 px	Q	Aprovado	Aprovado	URL exata
3	500 × 500 px	H	Aprovado	Aprovado	URL exata

Resultado: nos três casos, os dois leitores recuperaram exatamente a URL de teste. Os arquivos PNG correspondentes estão incluídos no pacote de entrega.

5. Limitação importante do ambiente

A tentativa de acessar quickchart.io diretamente a partir do ambiente de execução desta sessão falhou por resolução DNS/rede. Por isso, os PNGs usados na matriz acima foram gerados localmente para validar tamanho, correção e leitura; não são apresentados como respostas recebidas da API QuickChart. O script entregue faz a chamada real à QuickChart e executa os mesmos dois testes quando rodado em um computador com internet e as dependências instaladas.

6. Critérios de aceitação

• QR Code contém a URL de teste exata: aprovado na validação local.
• Leitura em tela nos tamanhos 200, 300 e 500 px: aprovado nos dois leitores de software.
• Nenhum dado pessoal real: aprovado.
• Validação da resposta efetiva da API QuickChart: pendente de execução externa.

7. Entregáveis e dependências

Entregáveis: imagem QR de exemplo; três PNGs de teste; script Python de integração com QuickChart; este relatório. Dependências do sprint: QR-01 e QR-02 (conforme descrição da issue).

8. Como concluir a validação da API

Em um computador com internet, instalar as dependências com `pip install requests pillow opencv-python pyzbar` e executar `python teste_qr03_quickchart.py`. O script solicita os três tamanhos/níveis à QuickChart, salva os PNGs e exige que OpenCV e ZBar leiam a URL exata. Para cumprir literalmente o critério de dois dispositivos físicos, também é recomendável escanear a imagem em dois celulares/leitores diferentes.

Conclusão: a prova de conceito de codificação e leitura foi validada localmente. A chamada real à API e a verificação em dois dispositivos físicos permanecem como passos finais antes de marcar QR-03 como concluída.

Fonte técnica: QuickChart — QR Code API.

Apêndice — arquivos de implementação e evidências

Esta seção reúne dentro do mesmo PDF o código Python completo que também foi entregue separadamente e uma imagem de exemplo do QR Code.

Código Python completo: teste_qr03_quickchart.py

```
"""Prova de conceito QR-03: QuickChart QR API.

Execute com: python teste_qr03_quickchart.py
Dependencias: pip install requests pillow opencv-python pyzbar qrcode
A API QuickChart precisa estar acessivel pela internet.
"""
from pathlib import Path
from urllib.parse import urlencode
import requests
from PIL import Image
import cv2
from pyzbar.pyzbar import decode as zbar_decode

URL_ALVO = "https://example.com/qr-03"  # URL ficticia e segura para validacao
API = "https://quickchart.io/qr"
PASTA = Path("resultados_qr03")
PASTA.mkdir(exist_ok=True)
CASOS = [(200, "M"), (300, "Q"), (500, "H")]

for tamanho, correcao in CASOS:
    params = {"text": URL_ALVO, "size": tamanho, "ecLevel": correcao,
              "format": "png", "margin": 4, "dark": "000000", "light": "ffffff"}
    resposta = requests.get(API, params=params, timeout=30)
    resposta.raise_for_status()
    if not resposta.headers.get("content-type", "").startswith("image/png"):
        raise RuntimeError(f"Resposta inesperada da API: {resposta.headers.get('content-type')}")

    arquivo = PASTA / f"qr03_{tamanho}_{correcao}.png"
    arquivo.write_bytes(resposta.content)
    with Image.open(arquivo) as img:
        dimensoes = img.size

    # Leitor 1: OpenCV
    imagem_cv = cv2.imread(str(arquivo))
    texto_cv, pontos, _ = cv2.QRCodeDetector().detectAndDecode(imagem_cv)
    ok_cv = texto_cv == URL_ALVO

    # Leitor 2: ZBar (via pyzbar)
    leituras = zbar_decode(Image.open(arquivo))
    textos_zbar = [item.data.decode("utf-8") for item in leituras]
    ok_zbar = URL_ALVO in textos_zbar

    print(f"{arquivo}: dimensoes={dimensoes}; OpenCV={ok_cv}; ZBar={ok_zbar}")
    if not (ok_cv and ok_zbar):
        raise RuntimeError(f"Falha de leitura no caso {tamanho}px / EC {correcao}")

print("Todos os casos foram gerados pela API e decodificados pelos dois leitores.")
```

Imagem de exemplo do QR Code

QR Code local de demonstração com a URL fictícia https://example.com/qr-03. Esta imagem serve como evidência da validação local de leitura; não é uma imagem obtida diretamente da API QuickChart.



Para a conclusão da QR-03, execute o script em um computador com internet e valide a leitura em dois dispositivos físicos, conforme descrito no relatório técnico.